



23

Refactoring & structure projet

US3.1 · C6

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Transformer un notebook en package src/ importable et testé
- › Configurer pyproject.toml (deps, scripts) avec uv
- › Appliquer ruff/black + type hints ; config via pydantic-settings
- › Éviter la fuite de données et normaliser les IDs machines

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE pytest vert · ruff check . · indusense --help

1

Module 23 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Le notebook de modélisation fait 1 200 lignes et plus personne n'ose y toucher. Demain, un collègue doit réutiliser ton modèle : il doit pouvoir l'installer et l'appeler en deux lignes.

Angle éthique / risque (C2) Un code illisible rend les décisions de l'IA non auditable. Structurer le projet, c'est aussi rendre la solution traçable et vérifiable.

■ Contexte & outils — pourquoi ce module

Contexte : un modèle de notebook n'est ni testable ni réutilisable. Pourquoi : on le range en package pour le rendre fiable et partageable.

- package src/ : une responsabilité par dossier (data, features, models)
- pyproject.toml : déclare dépendances, Python (3.13) et la commande indusense
- uv + uv.lock : versions EXACTES figées (fini le « ça marche chez moi »)
- pytest : tests automatiques (dont l'anti-fuite temporelle)
- ruff / black : qualité et format du code, automatiquement

Objectif du module Un code importable, testé, reproductible : la fondation de tout le sprint.

■ Référence amont : à quel horizon prédire la panne ?

Le modèle hérité du Sprint 2 prédit une probabilité de panne. En amont, une équipe data-science a tranché un arbitrage clé — à quel horizon prédire ? — qu'on industrialise sans le refaire (chiffres : référence Marine).

- 6 h → panne 4,5 % · ROC-AUC 0,84 — facile, mais anticipe peu
- 12 h → panne 8,8 % · ROC-AUC 0,80
- 24 h → panne 16,6 % · ROC-AUC 0,78 — HORIZON RETENU (bon compromis)
- 48 h → panne 25,2 % · ROC-AUC 0,72 — anticipe loin, se sépare mal

→ Arbitrage modèle (24 h)

Baseline vs tuné « frugal » : PR-AUC quasi égale ($0,619 \approx 0,619$) → on garde le plus sobre comme candidat v1.

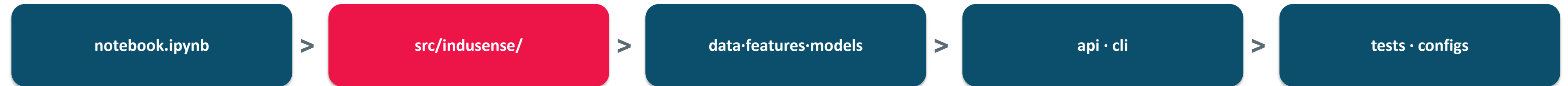
! Garde-fou chiffres

Chiffres de la RÉFÉRENCE Marine (15 machines, dataset HF) — PAS le gold étudiant (4 machines $\approx 10,5$ %) ni le flux complet ($\approx 4,78$ %).

EN UNE PHRASE Plus l'horizon est lointain, plus la panne est fréquente mais plus elle est dure à prédire : 24 h = le compromis retenu.

■ Du notebook au package : séparer les responsabilités

Un projet industriel range son code par responsabilité dans un package src/, pas dans un notebook fourre-tout.



L'analogie

Passer du notebook au package, c'est ranger un atelier en vrac en boîtes à outils étiquetées : on retrouve, on réutilise, on prête sans tout casser.



Le jargon décodé

Package = dossier importable. Module = un fichier .py. Refactoring = réorganiser sans changer le comportement.

EN UNE PHRASE On éclate le notebook en modules à responsabilité unique, importables et testables.

■ pyproject.toml & uv : la carte d'identité du projet

pyproject.toml déclare tout ce dont le projet a besoin ; uv l'installe à l'identique pour tout le monde.

- Dépendances + version de Python ($\geq 3.13, < 3.14$) + scripts (commande induSense)
- uv lit pyproject + un uv.lock = versions EXACTES figées
- Ta machine, la CI et la prod installent strictement les mêmes versions



Le saviez-vous ?

Le « ça marche chez moi » vient presque toujours de versions différentes. Un lockfile (uv.lock) tue ce bug : il fige jusqu'au patch.



Détail technique

Python 3.13 est la seule version supportée : pyproject.toml et uv.lock font foi sur chaque poste.

EN UNE PHRASE Reproductibilité = mêmes versions partout, garanties par pyproject + uv.lock.

■ La fuite de données : l'ennemi n°1

Si l'information du futur entre dans les données d'entraînement, le modèle « triche » et son score ment.

Split aléatoire = FUIITE

- passé + futur mélangés
- score faussement excellent
- échec en production

Split temporel par machine

- train = passé
- test = futur
- score honnête

! Attention — piège

Un score TROP beau (AUC 0,99) doit alerter, pas réjouir : c'est souvent une fuite de données.

→ Cas réel InduSense

On trie par (machine, timestamp) puis on coupe au point temporel ; un test vérifie qu'aucune ligne future n'entre dans le train.

EN UNE PHRASE Découpage temporel par machine : non négociable en maintenance prédictive.

■ Normaliser les identifiants : parler une seule langue

Les capteurs nomment les machines de plusieurs façons ; on canonicalise vers une seule forme.



? Le saviez-vous ?

Formats vérifiés : MACH-01, MACH_01, M-06, M-2. Le référentiel complet compte 15 machines ; le nombre total de variantes brutes n'est pas sourcé.

+ Astuce de pro

Normalise au bord (entrée des données ET de l'API) : le cœur du code ne voit qu'une seule forme.

EN UNE PHRASE Nettoyer au plus tôt : une seule forme d'identifiant dans tout le système.

■ Du notebook au package : une seule source de vérité

Une cellule de notebook duplique la logique ; le package l'isole dans une fonction testée — une seule source de vérité.

```
# avant - cellule de notebook (fragile, dupliquee)
df["temp_lag1"] = df.groupby("machine_id")["temperature"].shift(1)
# apres - package (testee, reutilisable)
from indusense.features.temporal import add_temporal_features
df = add_temporal_features(df)
```

→ Tour du squelette

temporal.py (shift avant rolling) · loaders.py (normalize_machine_id) · tests/ · cli.py
(train/predict) · pyproject.toml.

+ Chaque chose à sa place

data/ charge · features/ transforme · models/ entraîne · api/ expose · cli orchestre.

EN UNE PHRASE On code la transformation une seule fois, dans features/, et on l'appelle partout.

■ pyproject.toml : la carte d'identité technique du projet

pyproject.toml répond à 4 questions : quoi, avec quel Python et quelles dépendances, comment l'installer, quelles commandes et outils qualité.

```
[project]
name = "indusense"
requires-python = ">=3.13,<3.14"
dependencies = ["pandas>=2.2", "scikit-learn>=1.5", "..."]
[project.optional-dependencies]
dev = ["pytest", "ruff", "black"]
[project.scripts]
indusense = "indusense.cli:main"
```



Le saviez-vous ?

[project.scripts] crée la commande indusense : la taper appelle main() dans cli.py.



Détail technique

[project.optional-dependencies] dev = les outils d'atelier (tests, qualité), pas nécessaires en prod.

EN UNE PHRASE Un pyproject propre = n'importe qui installe, teste et exécute le projet de la même façon.

■ Préflight poste (Windows) : c'est 3.13 qui fait foi

Le Python global peut afficher 3.14 — ce n'est pas grave. Seul le venv du projet fait foi.

```
winget install --id astral-sh.uv -e # si uv absent
uv venv --python 3.13
uv sync --frozen --extra dev
uv run python --version # doit afficher 3.13.x
```

i python 3.14 ?

Ce qui fait foi, c'est `uv run python --version (3.13.x)` dans le projet, pas le python global.

! Docker dès J1

Vérifie `docker run hello-world` aujourd'hui — sinon 45 min perdues au J3.

EN UNE PHRASE La preuve d'environnement, c'est `uv run python --version` dans le projet : 3.13.x.

■ À retenir en 3 points

- Un package s'installe et s'importe — fini le copier-coller de cellules.
- Les chemins passent par pathlib : le code tourne sur PC et Mac.
- Les tests figent le comportement AVANT de refactorer.

2

Module 23 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — refactorer en package

Micro-étapes : on complète des trous, on corrige à chaque étape.

```
# arborescence cible
src/indusense/{data,features,models}/
tests/      pyproject.toml      .env.example
```

```
uv sync --frozen --extra dev
uv run pytest -q          # suite collectée ; 0 échec
uv run ruff check .       # propre
```

■ TP — CLI & normalisation des IDs

- Exposer une CLI : `indusense train` / `indusense predict`
- Activité dédiée : `normalize_machine_id`

```
# MACH-01 / MACH_01 / M-06 / M-2  ->  MACH-%02d
uv run indusense --help           # la CLI répond
```

✓ **Preuve attendue** `pytest` vert · `ruff` propre · `indusense --help` affiche `train/predict`.

■ TP — extraire une fonction propre (code nouveau)

On crée une fonction testée, au bon endroit : cleaning.py impute les manques par machine, et son test le prouve.

```
# src/indusense/features/cleaning.py
def clean_sensor_data(df, cols=("temperature", "pressure_bar")):
    df = df.drop_duplicates().copy()
    for c in cols:
        df[c] = df.groupby("machine")[c].transform(
            lambda s: s.fillna(s.median()))
    return df
```

```
uv run pytest tests/test_cleaning.py -q
uv run ruff check .
```

Preuve imputation par machine validée par le test ; ruff propre.

3

Module 23 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — durcir le package

Avancez à votre rythme. Le formateur reste disponible.

- Ajouter un test unitaire sur une fonction de feature engineering
- OU configurer pre-commit (ruff + black) en local
- Vérifier : le commit déclenche-t-il bien le hook ?

Posture en distanciel

Si vous bloquez, Appelez le formateur en visio ou par chat ; on débriefe les trouvailles en groupe à la fin de la séquence.

■ Definition of Done & transition

- DoD : package importable, pytest vert, ruff propre, CLI ok
- Livable : src/indusense/ + model_metadata.json

Transition → Module 24 On automatise tout ça en CI (GitHub Actions) + versioning data/modèle.